

Overview – Testing for `IllegalArgumentException`

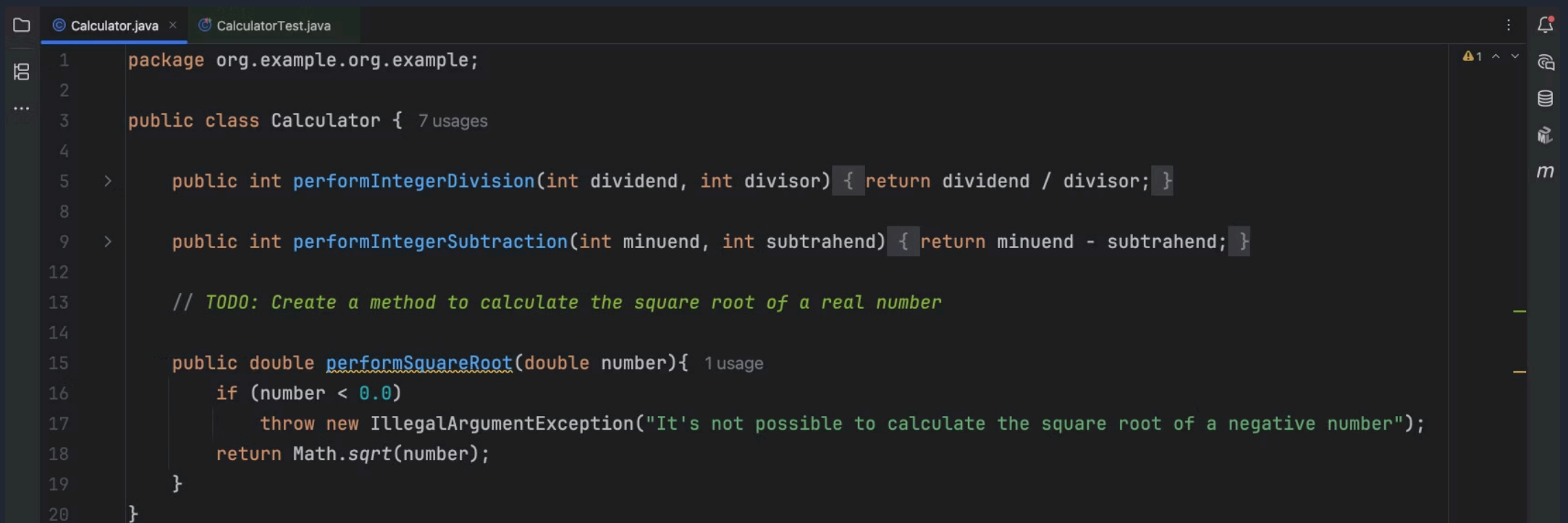
This **lesson** presents the **solution** to the **previous practice exercise**. The task involved **adding** a **new square-root method** to the `Calculator` **class**. You also needed to write a **JUnit test** to **verify** that a **negative input triggers** an **exception**. The goal is to reinforce **exception testing** using `assertThrows`.

 **by Rodrigo Martins Pagliares**

Last update: 11/27/2025



Step 1 – Creating the Square Root Method



```
1 package org.example.org.example;
2
3 public class Calculator { 7 usages
4
5     > public int performIntegerDivision(int dividend, int divisor) { return dividend / divisor; }
6
7
8     > public int performIntegerSubtraction(int minuend, int subtrahend) { return minuend - subtrahend; }
9
10
11
12
13     // TODO: Create a method to calculate the square root of a real number
14
15     public double performSquareRoot(double number){ 1 usage
16         if (number < 0.0)
17             throw new IllegalArgumentException("It's not possible to calculate the square root of a negative number");
18         return Math.sqrt(number);
19     }
20 }
```

Exception Handling

The `performSquareRoot(double number)` **method** must throw `IllegalArgumentException` for negative values.

Valid Input Processing

It returns `Math.sqrt(number)` when the input is valid.

Safe Design

This design ensures the **method handles invalid input explicitly** and safely.

Clear Messaging

The **exception message** must be **clear** to support **debugging** and **unit testing**.

Square Root Method – Code Snapshot

```
public double performSquareRoot(double number){ 1 usage
    if (number < 0.0)
        throw new IllegalArgumentException("It's not possible to calculate the square root of a negative number");
    return Math.sqrt(number);
}
```

A quick look at the expected implementation:

- Check if the input number is negative.
- Throw a meaningful exception if so.
- Otherwise delegate to Math.sqrt.

This structure supports predictable and testable behavior.

Step 2 – Creating the JUnit Test

01

Verify Exception Throwing

The test verifies that calling `performSquareRoot` with a negative input throws `IllegalArgumentException`.

02

Readable Annotations

The `@DisplayName` annotation makes the test more readable.

03

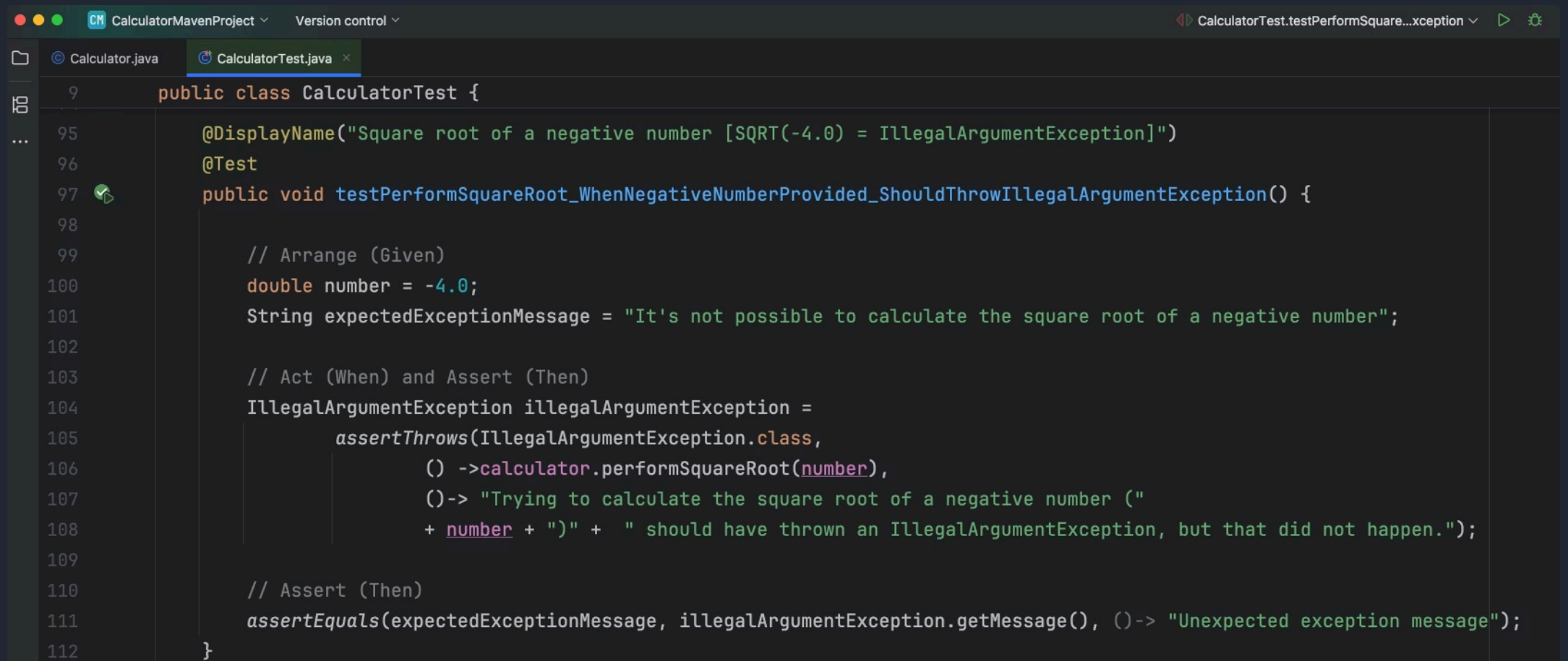
Test Pattern

The test follows **Arrange** → **Act & Assert** → **Assert** pattern.

04

Exception Capture

Using `assertThrows` allows capturing and inspecting the **thrown exception**.



```
9      public class CalculatorTest {
95         @DisplayName("Square root of a negative number [SQRT(-4.0) = IllegalArgumentException]")
96         @Test
97         public void testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException() {
98
99             // Arrange (Given)
100            double number = -4.0;
101            String expectedExceptionMessage = "It's not possible to calculate the square root of a negative number";
102
103            // Act (When) and Assert (Then)
104            IllegalArgumentException illegalArgumentException =
105                assertThrows(IllegalArgumentException.class,
106                    () -> calculator.performSquareRoot(number),
107                    () -> "Trying to calculate the square root of a negative number ("
108                        + number + ")" + " should have thrown an IllegalArgumentException, but that did not happen.");
109
110            // Assert (Then)
111            assertEquals(expectedExceptionMessage, illegalArgumentException.getMessage(), () -> "Unexpected exception message");
112        }
```

Inside the Test – Arrange

Setting Up Test Preconditions

```
public class CalculatorTest {  
    @DisplayName("Square root of a negative number [SQRT(-4.0) = IllegalArgumentException]")  
    @Test  
    public void testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException() {  
  
        // Arrange (Given)  
        double number = -4.0;  
        String expectedExceptionMessage = "It's not possible to calculate the square root of a negative number";  
    }  
}
```

- Define the invalid input, e.g., number = -4.0.
- Set the **expected exception message** to compare later.
- Prepare any required objects (e.g., **calculator instance** in `@BeforeEach` method).
- The **Arrange** section clarifies the preconditions for the test.

Inside the Test – Act & Assert

```
public class CalculatorTest {  
    public void testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException() {  
  
        // Act (When) and Assert (Then)  
        IllegalArgumentException illegalArgumentException =  
            assertThrows(IllegalArgumentException.class,  
                () -> calculator.performSquareRoot(number),  
                () -> "Trying to calculate the square root of a negative number (" +  
                    + number + ") " + " should have thrown an IllegalArgumentException, but that did not happen.");  
    }  
}
```

1

Define Exception Type

Use `assertThrows` with the expected exception type.

2

Execute Method Call

Call `calculator.performSquareRoot(number)` inside the lambda.

3

Automatic Failure

If the method **does not throw**, the test automatically **fails**.

4

Capture Exception

The **returned exception object** is **stored** for **further inspection**.

Inside the Test – Assert Message

```
// Assert (Then)  
assertEquals(expectedExceptionMessage, illegalArgumentException.getMessage(), ()-> "Unexpected exception message");
```

Message Comparison

Compare the actual message with the expected one using `assertEquals`.

Correct Exception Validation

This ensures the method not only throws an exception but throws it correctly.

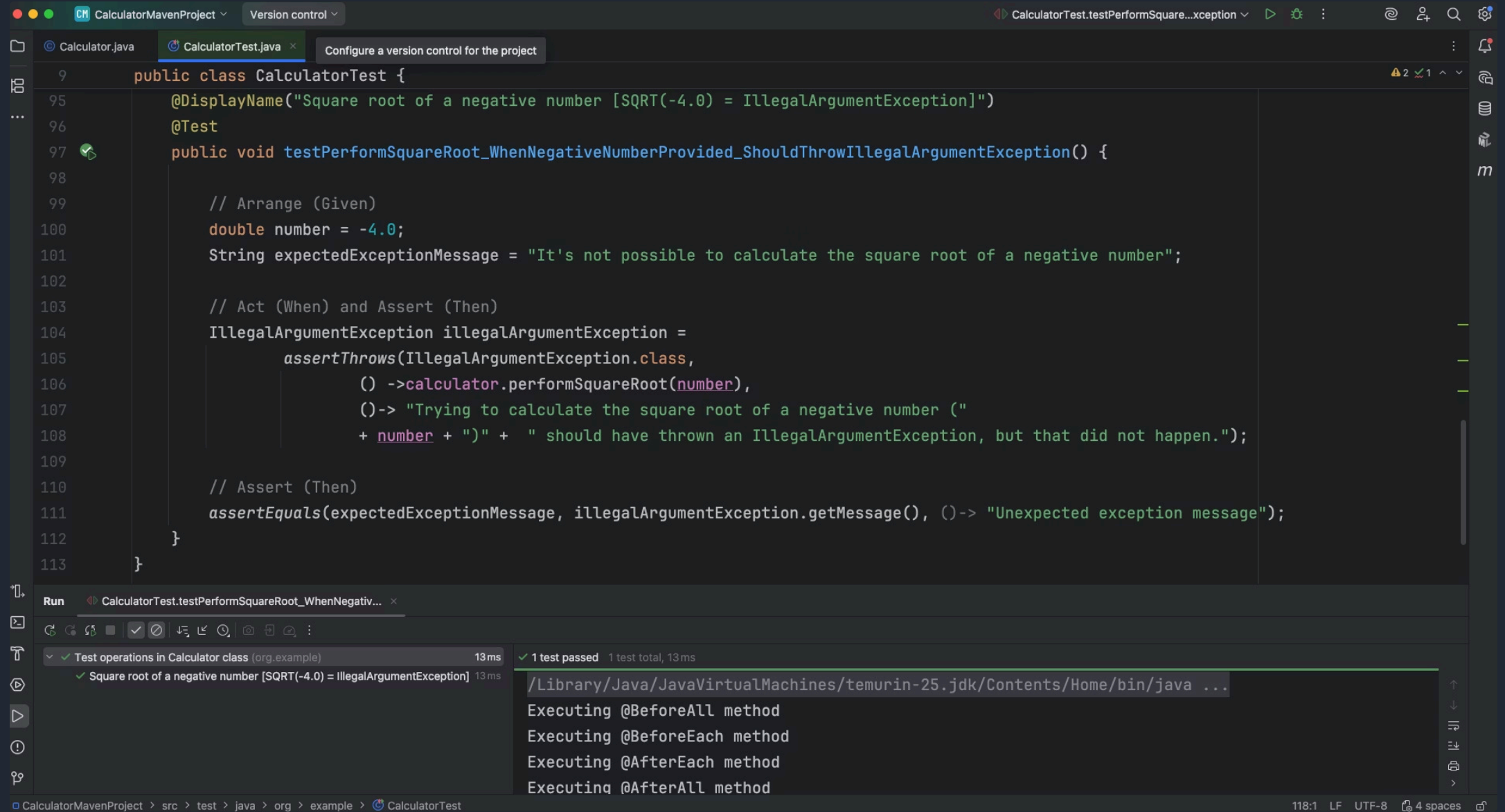
Debugging Support

A clear message improves debugging for future developers.

Final Validation

This step finalizes the validation of exception behavior.

Run the test method



The screenshot shows an IDE window with a Java file named `CalculatorTest.java`. The code defines a test method `testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException()` that uses JUnit annotations `@DisplayName` and `@Test`. The test method includes assertions for an `IllegalArgumentException` with a specific message. Below the code editor, the 'Run' tab is active, showing a successful test run. The test summary indicates '1 test passed' and '1 test total, 13 ms'. The test output shows the execution of lifecycle methods: `@BeforeAll`, `@BeforeEach`, `@AfterEach`, and `@AfterAll`.

```
9 public class CalculatorTest {
95     @DisplayName("Square root of a negative number [SQRT(-4.0) = IllegalArgumentException]")
96     @Test
97     public void testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException() {
98
99         // Arrange (Given)
100         double number = -4.0;
101         String expectedExceptionMessage = "It's not possible to calculate the square root of a negative number";
102
103         // Act (When) and Assert (Then)
104         IllegalArgumentException illegalArgumentException =
105             assertThrows(IllegalArgumentException.class,
106                 () -> calculator.performSquareRoot(number),
107                 () -> "Trying to calculate the square root of a negative number ("
108                     + number + ")" + " should have thrown an IllegalArgumentException, but that did not happen.");
109
110         // Assert (Then)
111         assertEquals(expectedExceptionMessage, illegalArgumentException.getMessage(), () -> "Unexpected exception message");
112     }
113 }
```

Run CalculatorTest.testPerformSquareRoot_WhenNegativ... x

Test operations in Calculator class (org.example) 13 ms

✓ 1 test passed 1 test total, 13 ms

✓ Square root of a negative number [SQRT(-4.0) = IllegalArgumentException] 13 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

Executing @BeforeAll method

Executing @BeforeEach method

Executing @AfterEach method

Executing @AfterAll method

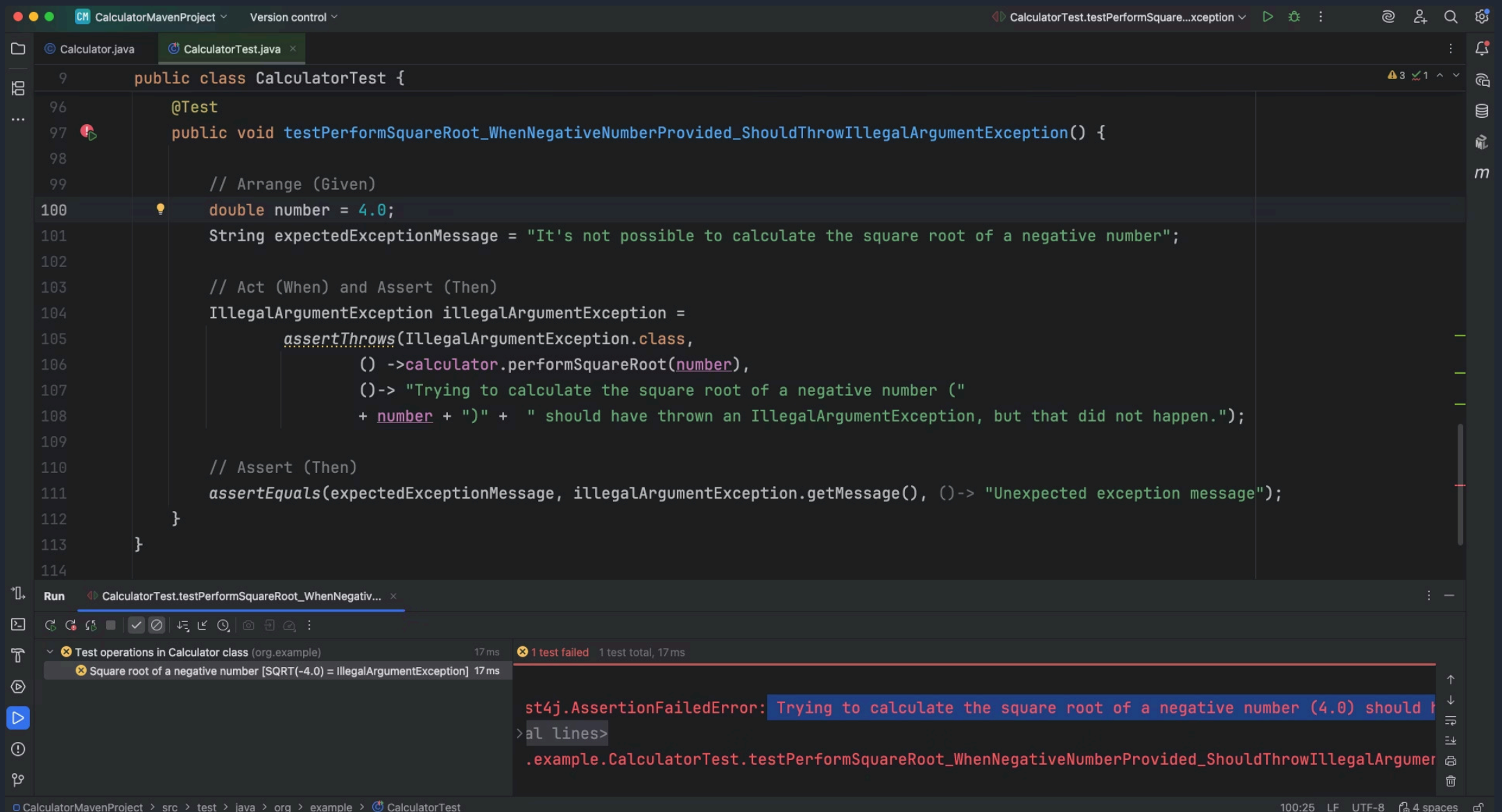
CalculatorMavenProject > src > test > java > org > example > CalculatorTest 118:1 LF UTF-8 4 spaces

Test Passed!

Making the test fail

Use a positive number

```
double number = 4.0;
```



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The main editor displays the code for 'CalculatorTest.java'. The test method 'testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException()' is shown. It sets 'number' to 4.0 and expects an 'IllegalArgumentException' with a specific message. The test fails because the actual exception message is 'Trying to calculate the square root of a negative number (4.0) should have thrown an IllegalArgumentException, but that did not happen.', which does not match the expected message.

```
9      public class CalculatorTest {
96         @Test
97         public void testPerformSquareRoot_WhenNegativeNumberProvided_ShouldThrowIllegalArgumentException() {
98
99             // Arrange (Given)
100            double number = 4.0;
101            String expectedExceptionMessage = "It's not possible to calculate the square root of a negative number";
102
103            // Act (When) and Assert (Then)
104            IllegalArgumentException illegalArgumentException =
105                assertThrows(IllegalArgumentException.class,
106                    () -> calculator.performSquareRoot(number),
107                    () -> "Trying to calculate the square root of a negative number ("
108                        + number + ") should have thrown an IllegalArgumentException, but that did not happen.");
109
110            // Assert (Then)
111            assertEquals(expectedExceptionMessage, illegalArgumentException.getMessage(), () -> "Unexpected exception message");
112        }
113    }
114
```

The Run window at the bottom shows the test results. The test 'Square root of a negative number [SQRT(-4.0) = IllegalArgumentException]' failed. The error message is: 'st4j.AssertionFailedError: Trying to calculate the square root of a negative number (4.0) should have thrown an IllegalArgumentException, but that did not happen.'.

Conclusion



Exception Testing Mastery

You learned how to implement and **test exception-throwing** behavior in **JUnit**.



Input Validation

The exercise demonstrated **handling invalid input** and **verifying error messages**.



Essential Skills

Mastering `assertThrows` is essential for writing reliable **unit tests**.



Practice Complete

This completes the practice cycle for **exception testing**.



Open Floor Discussion

Questions to Consider

- Should the **square-root method** ever return a special value instead of throwing an exception?
- How would you **extend** the **test** to handle very large or edge-case inputs?
- Could **custom exception types** improve clarity in more complex applications?

